# Handling Missing Data: Random Sample Imputation, Missing Indicator and Automatic Imputer Selection

## Contents


1. Overview . . . 2
2. Random sample imputation . . . 3
   - 2.1 The idea . . . 3
   - 2.2 Why the distribution stays the same . . . 3
   - 2.3 Advantages . . . 5
   - 2.4 Disadvantages . . . 5
3. Random sample imputation on Titanic ages . . . 6
   - 3.1 The data and the fill . . . 6
   - 3.2 Distribution, variance and outliers . . . 6
   - 3.3 Covariance with other columns . . . 7
4. Same input, same fill . . . 8
   - 4.1 The problem . . . 8
   - 4.2 The fix: one seed per row . . . 8
5. Random sample imputation on categorical columns . . . 9
   - 5.1 The data . . . 9
   - 5.2 Check 1: category shares . . . 10
   - 5.3 Check 2: the target per category . . . 10
6. Missing indicator . . . 12
   - 6.1 The idea . . . 12
   - 6.2 Why it can help . . . 12
   - 6.3 On the Titanic data . . . 13
   - 6.4 The shortcut: `add_indicator=True` . . . 15
7. Choosing the imputer automatically with grid search . . . 16
   - 7.1 The idea . . . 16
   - 7.2 The pipeline . . . 16
   - 7.3 Naming the settings . . . 18
   - 7.4 The result where the fill matters: house prices . . . 18
   - 7.5 When every imputer ties: the Titanic pipeline . . . 19
8. Summary . . . 22
9. Sources . . . 23
10. Key terms . . . 23

**Where this fits:** Pipeline map step 4 (Clean). See the Course map.

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 Foundations | 1 Frame | 2 Get data | 3 Understand | **4 Clean** | 5 Features | 6 Reduce |
| 7 Split | 8 Model | 9 Evaluate | 10 Tune | 11 Deploy | 12 Test | 13 Monitor |

- **Builds on:** Missing values.
- **Leads to:** KNN imputer.
- **Compare with:** Simple imputation (mean, median, mode, constant).

## 1. Overview

**Key point:** Random sample imputation fills each gap with a real value drawn at random from the same column; a missing indicator adds a True/False column that marks where the gaps were.

Imputing numerical data filled gaps in numerical features with one fixed number, and imputing categorical data filled gaps in categorical features with the mode (the most common value) or the word "Missing". A **feature** (G-772) is an input variable (one column of the data table), the **target** (G-1949) is the output we predict, and an **observation** (G-1374) is one record (one row). This Note adds:

- random sample imputation (Sections 2 to 5);
- the missing indicator (Section 6);
- a way to let the computer pick the best technique for us, **grid search** (G-872) (Section 7).

Figure 1 shows the two techniques on a small `Age` column with two gaps:

- **Random sample imputation** (G-1616) puts the known values in a bag and draws one at random for each gap. The technique works for numerical and categorical features alike.
- **Missing indicator** (G-1233) keeps the gaps for another imputer to fill, and adds a new column, `Age_NA`, that is True where the value was missing.

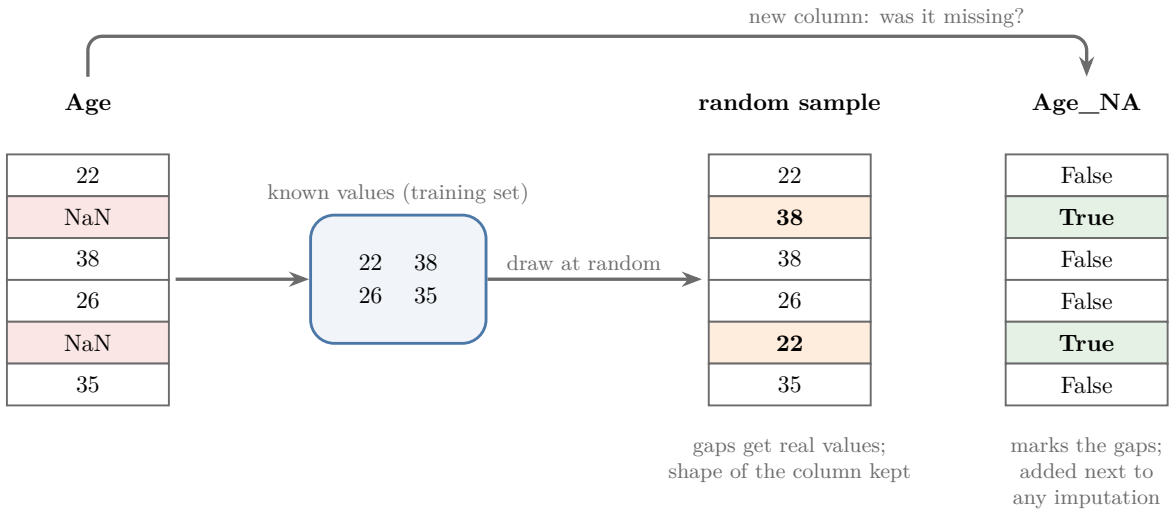



**Figure 1:** Random sample imputation draws each fill value from the known values; the missing indicator adds a column marking the gaps

Both techniques look only at the feature with the gap, never at the other features. Such a method is called **univariate imputation** (G-2051). The last section uses grid search to choose the imputation strategy automatically, together with the model's own settings.

## 2. Random sample imputation

> **Key point:** Each missing value is replaced by a value picked at random from the values the column already has.

### 2.1 The idea

> **Key point:** We do not invent numbers; we reuse the column's own values, chosen at random.

Random sample imputation (G-1616) replaces every missing value of a column with a value drawn at random from the known values of the same column. If the known ages are 22, 38, 26 and 35, a gap becomes one of those four, never some other number such as 71 or 99.

The same works for a categorical column. If a `gender` column holds "male" and "female" plus some gaps, each gap becomes "male" or "female", drawn at random from the known rows.

### 2.2 Why the distribution stays the same

> **Key point:** Common values are drawn often and rare values rarely, so the filled values follow the same shape as the known ones.

Think of every known value written on a slip of paper and put into one bag. Values that occur often have many slips, so a random draw lands on them often. Rare values have few slips and are drawn rarely.

The chance of drawing a value, step by step:

1. **In words:** count how many known values fall in a range, and divide by the number of known values.
2. **Formula:**
$$P(\text{in range}) = \frac{n_{\text{range}}}{n_{\text{known}}}$$
Here $P(\text{in range})$ is the chance that a random draw lands in the range, $n_{\text{range}}$ the known values in the range and $n_{\text{known}}$ all known values.
3. **Example:** in the Titanic training set of Section 3, 174 of the 564 known ages are in their twenties:
$$\frac{174}{564} = 0.309.$$
So about 31% of the 148 gaps, around 46 of them, get an age in the twenties, the same share as in the known data. Only 20 known ages are 60 or over, so very few gaps get such an age.

Figure 2 runs the draws for the 148 gaps of Section 3. Watch the orange bars: after a few draws they jump around, but as the draws add up they settle into the grey outline of the known ages; 49 of the 148 land in the twenties.

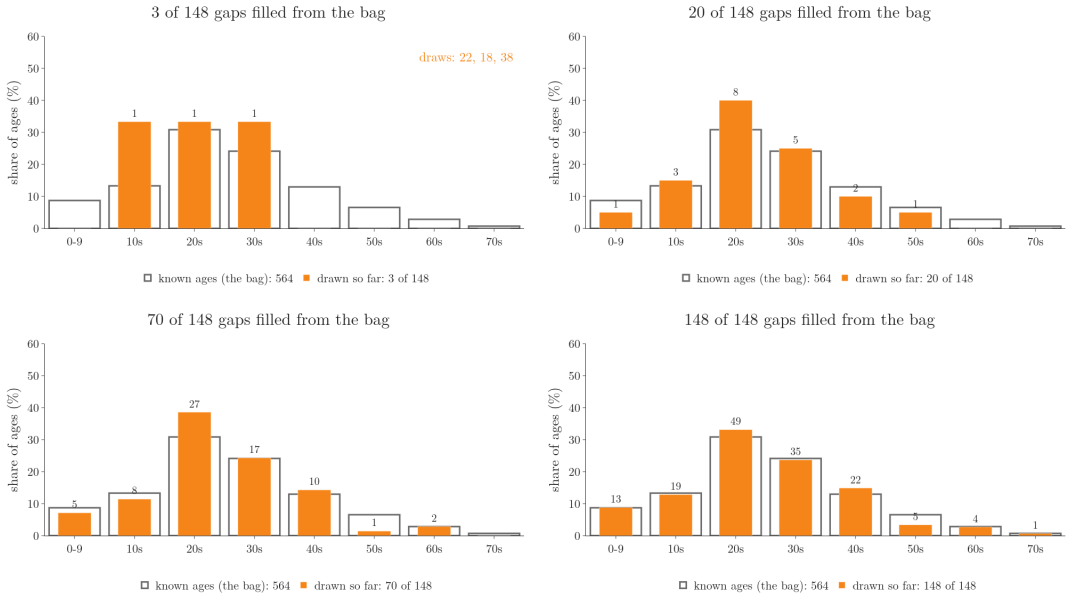



**Figure 2:** Random sample imputation of the 148 missing training ages: the share of drawn ages per decade (orange) settles onto the share among the 564 known ages (grey outline)

Because every range gets its fair share of the filled values, two things stay almost the same: the **distribution** (G-626), which is how the values spread over their range, and the **variance**

(G-2074), which is the average squared distance of the values from their mean. **Mean imputation** (G-1197; mean and median imputation) instead piles every filled value at one point.

### 2.3 Advantages

> **Key point: Random sample imputation** (G-1616) is easy to apply and keeps the distribution and variance intact, which suits linear models.

1. **Easy to apply.** One line of pandas per column.
2. **Keeps the distribution and the variance**, as Section 2.2 explains.
3. **Good for linear models**, such as linear and **logistic regression** (G-1120). Filling every gap with one value, such as the mean, shrinks the variance and weakens the links between columns, and this biases almost every estimate except the mean (van Buuren, FIMD §1.3.3). A linear model's coefficients are such estimates. Random sample imputation keeps the variance: Section 3.2 shows 200.03 against the original 204.35, while **mean imputation** (G-1197) gives 161.81.

**Tree-based algorithms** (G-2013), such as decision trees, gain less from it: the random values add noise to the data. The intuition: a tree splits on the order of the values, so keeping the exact shape of the **distribution** (G-626) matters less to it.

> **Python:** scikit-learn has no random sample imputer, so we write it in pandas.
> ```
> pool = X_train["Age"].dropna()         # the bag
> gaps = X_train["Age"].isnull()
> X_train.loc[gaps, "Age"] = pool.sample(
>     gaps.sum(), random_state=42).values
> ```
> **`sample(n)`** (G-140) draws `n` values at random from a Series. **`random_state`** (G-128) fixes the draw, so the same code gives the same values every time. **`.values`** drops the row labels of the drawn values; without it, pandas would try to match them to the gaps' labels.

### 2.4 Disadvantages

> **Key point: Random sample imputation** (G-1616) weakens the feature's relationships with other features, adds randomness, and needs the training data at prediction time.

1. **The covariance with other features changes.** The **covariance** (G-496) measures how two features move together: positive when they rise together. Each filled value is drawn without looking at the rest of the **observation** (G-1374), so the feature's link to other features weakens.
2. **The fill is random.** Run the code twice without a fixed **random seed** (G-1617), the number that fixes a random draw, and the filled values differ.
3. **Production needs memory.** A gap in new data must be filled from the training values, so the deployed model has to keep the whole training column on the server. For a large training set, the stored column takes a lot of memory.

Random sample imputation keeps the shape however many values are missing. Section 5 shows a check that can still fail when many values are missing.

## 3. Random sample imputation on Titanic ages

> **Key point:** On the Titanic data, **random sample imputation** (G-1616) of `Age` keeps the density curve and the **variance** (G-2074) almost unchanged, but the covariance with `Fare` drops.

### 3.1 The data and the fill

> **Key point:** We split first, and draw the fill values for both sets from the known training ages.

The data is Kaggle's Titanic training file, 891 passengers. We use `Age` and `Fare` as inputs and `Survived` as the **target** (G-1949). `Age` has 177 missing values (19.9%); `Fare` has none.

After an 80/20 split, the training set has 712 rows, 148 of them with no age (20.8%). The test set has 179 rows, 29 of them with no age.

Both sets are filled from the same bag: the 564 known training ages. Drawing test values from the test set itself would let test data shape the inputs, which is **data leakage** (G-535; information from the test set getting into the inputs, see the toy project).

> **Python:** One helper for both sets.
```python
def random_fill(col, pool, seed):
    out = col.copy()
    gaps = out.isnull()
    out[gaps] = pool.sample(gaps.sum(),
                            random_state=seed).values
    return out

pool = X_train["Age"].dropna()
X_train["Age_imputed"] = random_fill(X_train["Age"], pool, 42)
X_test["Age_imputed"] = random_fill(X_test["Age"], pool, 42)
```
We keep the original column next to the filled one, to compare them. The seed 42 makes the numbers below repeatable; another seed gives slightly different numbers.

### 3.2 Distribution, variance and outliers

> **Key point:** The density curve and the box barely move, and the **variance** (G-2074) changes from 204.35 to 200.03.

Figure 3 compares the original `Age` (blue) with the random sample imputed one (orange). For contrast, it also shows mean imputation (green dashed; **mean imputation** (G-1197)).

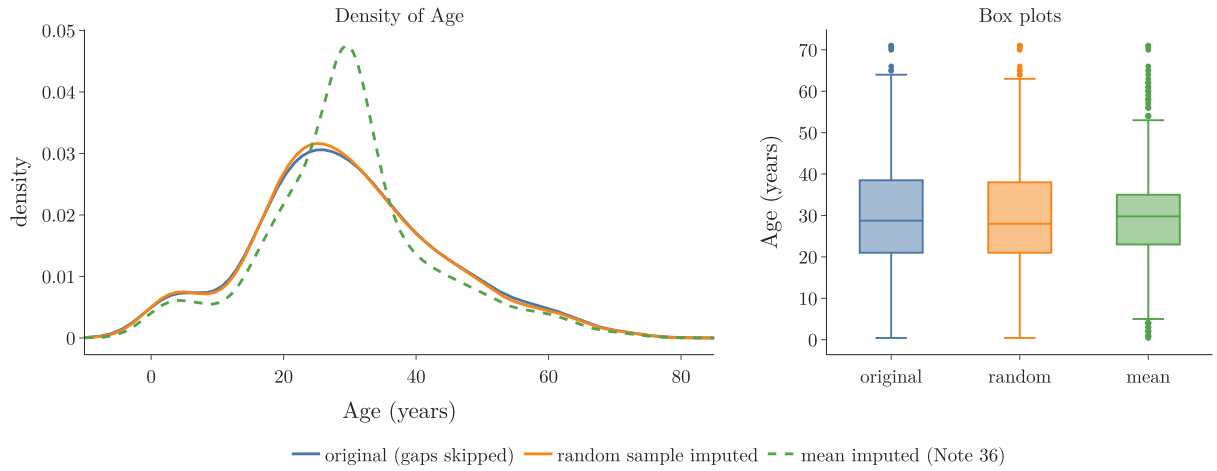



**Figure 3:** Age before and after imputation: the random sample curve lies on the original, while mean imputation creates a tall peak and many new outliers

The orange curve lies almost on top of the blue one. The box plots agree (a box plot draws the middle half of the values as a box, and marks far-out values as outliers): the box runs from 21 to 38.25 before and from 21 to 38 after, and the outliers go from 7 to 10. Mean imputation, by contrast, narrows the box to 23 to 35.

| `Age` in the training set | Original | Random sample | Mean imputation |
|---|---|---|---|
| Variance | 204.35 | 200.03 | 161.81 |
| Mean | 29.79 | 29.58 | 29.79 |
| Median | 28.75 | 28.00 | 29.79 |
| Outliers (box plot) | 7 | 10 | 69 |

### 3.3 Covariance with other columns

> **Key point:** The **covariance** (G-496) of `Age` with `Fare` falls from 71.51 to 53.26, because the drawn ages ignore the fare.

| Covariance with `Fare` | Value |
|---|---|
| `Age` original | 71.51 |
| `Age` random sample imputed | 53.26 |

A passenger with a high fare tended to be a little older. The drawn ages pay no attention to the fare, so for the 148 filled rows this pattern is lost, and the covariance drops by about a quarter. The drop is disadvantage 1 of Section 2.4.

## 4. Same input, same fill

> **Key point:** In production, seed the draw with a value from the row itself, so the same input always gets the same fill value and the same prediction.

### 4.1 The problem

> **Key point:** Without a fixed seed, the same user can get a different prediction each time they ask.

Suppose a website uses the Titanic model. A user enters a fare of 56.50 but leaves the age empty. The site draws an age at random, say 30, and shows a prediction.

The user presses the button again with the same input. This time the draw gives 4, and the prediction may change. The same question now has two answers, and the user loses trust in the model.

### 4.2 The fix: one seed per row

> **Key point:** Use a value from the row, such as its fare, as the **random seed** (G-1617).

A random draw with a fixed seed always gives the same result. So we take the seed from the row: here, the fare as a whole number. A seed taken from the row's own values is a **per-row seed** (G-1479). The same fare always gives the same seed, so the same draw, so the same age.

> **Python:** Seeding each row's draw with its fare.

```python
def fill_one(row, pool):
    if pd.isna(row["Age"]):
        seed = int(row["Fare"])
        return pool.sample(1, random_state=seed).iloc[0]
    return row["Age"]

new = pd.Series({"Age": np.nan, "Fare": 56.4958})
[fill_one(new, pool) for _ in range(3)]   # [34.0, 34.0, 34.0]
```

Without the seed, `pool.sample(1)` three times gave 30, 4 and 27. Figure 4 asks eight times. Watch the two lines: the draw seeded by the fare stays at 34, while fresh draws (here 8 different seeds) jump between 17 and 52. **`int()`** turns a number into a whole number by dropping the decimals; `random_state` needs a whole number.

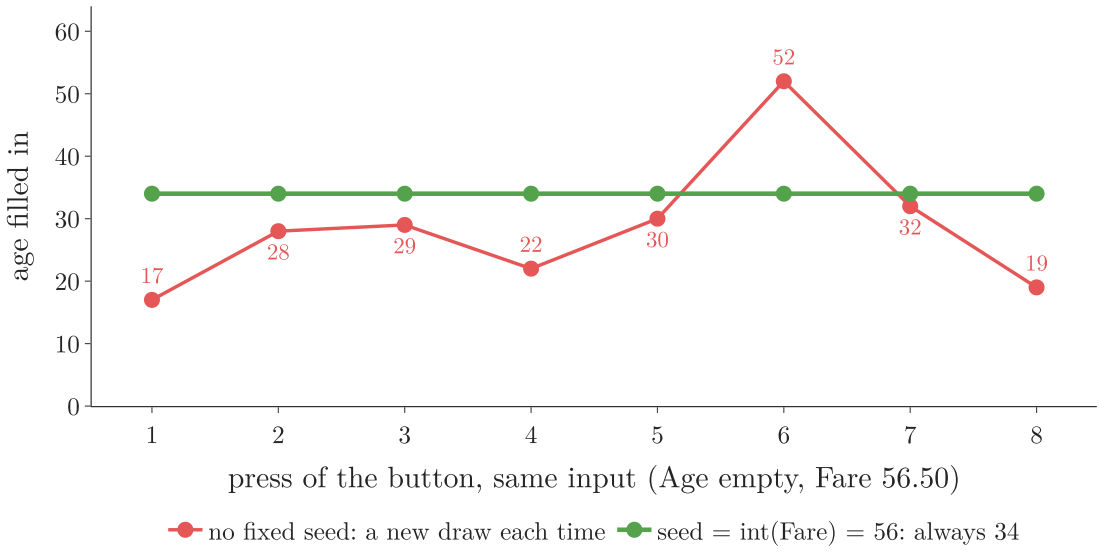



**Figure 4:** The same input filled eight times: a fresh draw each time (red) against a draw seeded by the row's fare (green)

> **Extra:** The seed should come from inputs that identify the row well. `int(row["Fare"])` drops the decimals, so fares 7.25 and 7.93 share the seed 7, and every passenger with that fare and no age gets the same age. Using several inputs together, for example a hash of the whole row, spreads the seeds better.

## 5. Random sample imputation on categorical columns

> **Key point:** On two house-quality columns, the category shares stay the same after **random sample imputation** (G-1616), but in `FireplaceQu` the sale price per category changes a lot.

### 5.1 The data

> **Key point:** The same two columns as in the house price data: `GarageQual` with 5.5% missing and `FireplaceQu` with 47.3% missing.

The data is the house price file of imputing categorical data: `GarageQual`, `FireplaceQu` and the **target** (G-1949) `SalePrice` for 1,460 houses. The full Kaggle file has 81 columns; `data/house_prices.csv` keeps these three.

We split 80/20 with the same seed as before. The training set has 1,168 houses: 65 with no `GarageQual` (5.6%) and 557 with no `FireplaceQu` (47.7%). Each column is filled from its own known training values.

### 5.2 Check 1: category shares

> **Key point:** Each category keeps its share, even in `FireplaceQu`, where almost half the values were drawn.

Figure 5 compares each category's share before (among the known values) and after (among all rows).

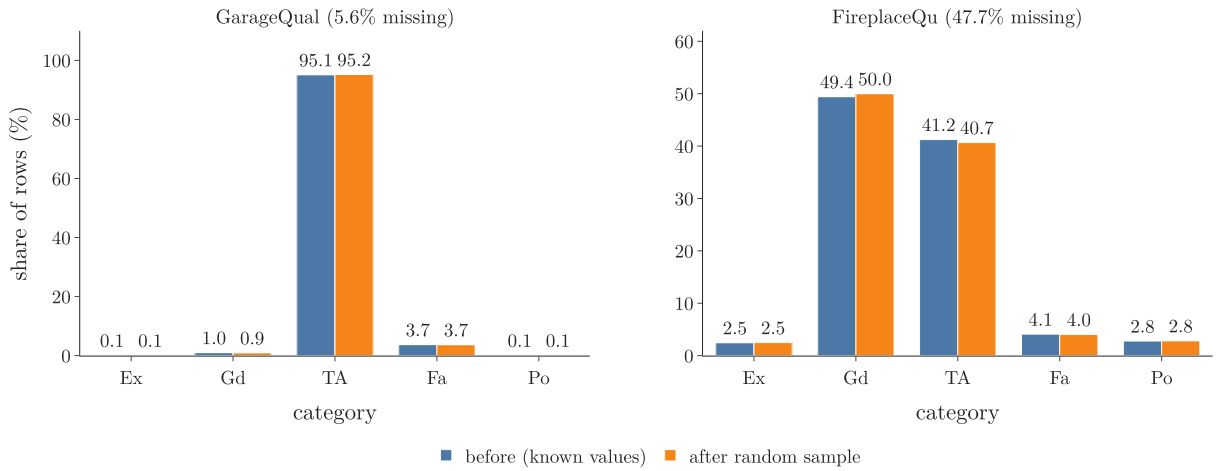



**Figure 5:** Category shares before and after random sample imputation: no share moves by more than 0.6 percentage points

| | `GarageQual` | | `FireplaceQu` | |
|---|---|---|---|---|
| Category | before | after | before | after |
| Gd | 1.0% | 0.9% | 49.4% | 50.0% |
| TA | 95.1% | 95.2% | 41.2% | 40.7% |
| Fa | 3.7% | 3.7% | 4.1% | 4.0% |
| Po | 0.1% | 0.1% | 2.8% | 2.8% |
| Ex | 0.1% | 0.1% | 2.5% | 2.5% |

The equal shares are the categorical version of Section 2.2: a category with half the slips gets about half the draws. Compare mode imputation, where Gd in `FireplaceQu` jumped to 72.9%.

### 5.3 Check 2: the target per category

> **Key point:** In `FireplaceQu`, cheap houses were spread into every category, so Gd and Ex houses now look much cheaper; this is a red flag.

The shares only tell part of the story. A second check compares the **target** (G-1949), `SalePrice`, for each category before and after (Figure 6).

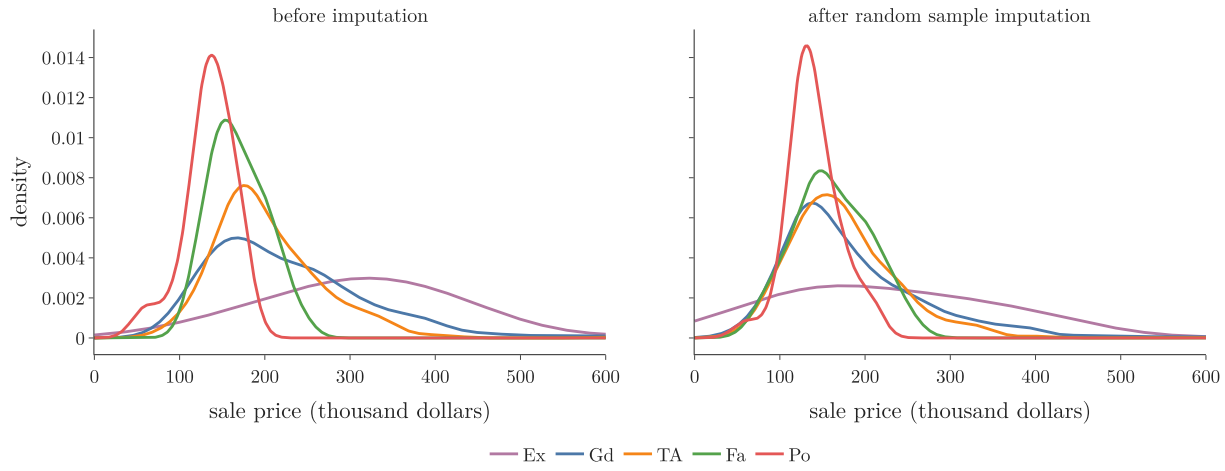



**Figure 6:** Sale price density per FireplaceQu category, before and after random sample imputation: the curves of Ex, Gd and TA shift left

The 557 houses with no `FireplaceQu` sold for a mean of 141,182 dollars, far below the Gd houses (225,668). The draws turned about half of them into Gd houses and two fifths into TA houses. Each category's prices moved towards the cheap ones:

| `FireplaceQu` | Mean price before | Mean price after |
|---|---|---|
| Ex | 337,145 | 240,230 |
| Gd | 225,668 | 184,529 |
| TA | 205,749 | 175,367 |
| Fa | 168,994 | 161,279 |
| Po | 135,112 | 139,591 |

Before imputation, a better fireplace meant a pricier house. After, Gd, TA and Fa houses look much more alike, so the link between fireplace quality and price is weaker in the imputed data. Such a change in the **distribution** (G-626) is a bigger problem for linear models.

For `GarageQual`, only 65 values were drawn, and the TA mean moves only from 186,766 to 182,093 dollars. So **random sample imputation** (G-1616) is fine for `GarageQual`, but not for `FireplaceQu`: too many values are missing there.

> **Extra:** As the house price data showed, a gap in `FireplaceQu` means "no fireplace": all 690 such houses have zero fireplaces. Drawing Gd or TA for them invents fireplaces, which explains the price shift. For gaps with a meaning, a "Missing" category or a **missing indicator** (G-1233) fits better.

## 6. Missing indicator

> **Key point:** A **missing indicator** (G-1233) adds a True/False column for each feature with gaps, so the model can learn whether "missing" itself matters.

### 6.1 The idea

> **Key point:** True where the value was missing, False where it was present.

A missing indicator (G-1233) is a new column added for each **feature** (G-772) that has gaps. The indicator holds True where that feature's value is missing and False where it is present. For `Age`, the new column `Age_NA` is True for the 148 passengers with no age (Figure 1).

The indicator does not fill the gaps. The original column is still imputed in the usual way, for example with the mean. The indicator is added next to it.

### 6.2 Why it can help

> **Key point:** The model can learn that **observations** (G-1374) with gaps behave differently from observations without.

The model gets a way to tell observations with a gap apart from observations without one. A doctor's form works the same way: a blank "smoker?" box can itself be a clue, so we note that it was left blank instead of guessing an answer. If the two groups behave differently, the model can use that difference.

On the Titanic training set, they do. Of the passengers with no recorded age, 28.4% survived; of those with an age, 39.2% survived (Figure 7). **Mean imputation** (G-1197) alone hides this: a filled age of 29.79 looks like any other age.

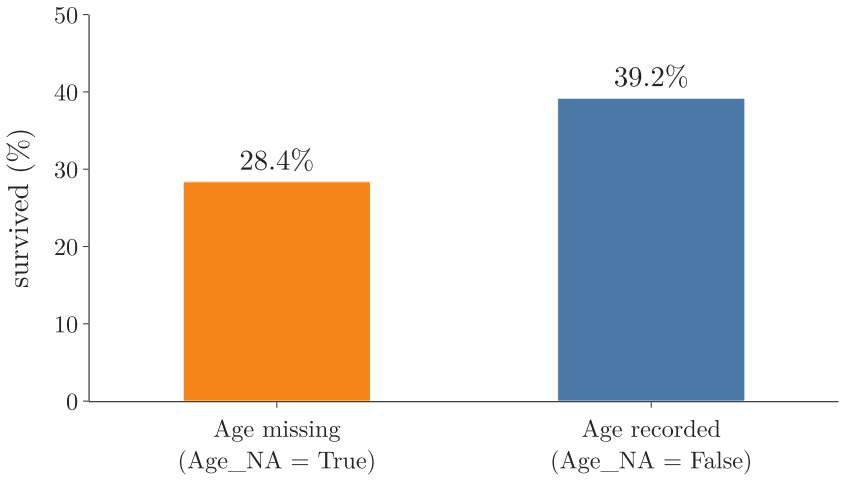



**Figure 7:** Survival in the Titanic training set for passengers with and without a recorded age: the gap the indicator lets the model see

The technique became popular through machine learning competitions, where adding indicators helped winning models. The indicator does not always help, but it is cheap to try when a model needs a few more points of **accuracy** (G-162).

> **Extra:** A **missing indicator** (G-1233) is most useful when the data is not missing completely at random: **MAR** (G-1158), where the gaps depend on another recorded **feature** (G-772), or MNAR, where they depend on the missing value itself (MCAR, MAR and MNAR). When gaps are pure chance, the indicator carries no information, and the model should give it a weight near zero.

### 6.3 On the Titanic data

> **Key point:** Adding `Age_NA` to mean-imputed `Age` and `Fare` lifts the test accuracy of logistic regression from 61.5% to 63.1%.

We train **logistic regression** (G-1120), a classification algorithm (logistic regression), on `Age` and `Fare` twice, with the same split as Section 3. The score is the **accuracy** (G-162): the fraction of test passengers classified correctly.

| Inputs | Test accuracy |
|---|---|
| `Age`, `Fare` (mean imputed) | 61.5% (110 of 179) |
| `Age`, `Fare` (mean imputed) + `Age_NA` | 63.1% (113 of 179) |

Three more test passengers are classified correctly.

The mechanism, step by step:

1. **In words:** the model multiplies each input by a learned number, its **coefficient** (G-407), adds the results to a score $z$, and turns $z$ into a chance between 0 and 1. The indicator is one more input, 1 for True and 0 for False, so it gets its own coefficient.
2. **Formula**, with the coefficients the model learned:

$$z = -0.419 - 0.019a + 0.016f - 0.295m$$

$$P(\text{survived}) = \frac{1}{1 + e^{-z}}$$

   Here $a$ is the age, $f$ the fare, and $m$ the indicator `Age_NA` (1 or 0).
3. **Example:** two passengers with the filled age 29.79 and a fare of 60. With a recorded age (`Age_NA` $= 0$), the chance is 49.7%:

$$z = -0.01$$

   With a missing age (`Age_NA` $= 1$), $z$ drops by 0.295 and the chance is 42.4%:

$$z = -0.01 - 0.295 = -0.31$$

The turning of $z$ into a chance is the **sigmoid** (G-1798): $z = 0$ gives 50%, a large positive $z$ gives a chance near 100%, a large negative $z$ a chance near 0%. For the second passenger:

$$\frac{1}{1 + e^{0.31}} = \frac{1}{1 + 1.36} = 0.42$$

The coefficient of `Age_NA` is negative ($-0.30$ after rounding): a missing age lowers the predicted chance of survival, as the survival rates in Section 6.2 suggest.

Figure 8 shows that coefficient at work. Two passengers have the same filled age, 29.79, and the same fare; only `Age_NA` differs. Watch the orange curve sit below the blue one at every fare, so the passenger with a missing age needs a higher fare (about 80 instead of 60) before the model predicts "survived".

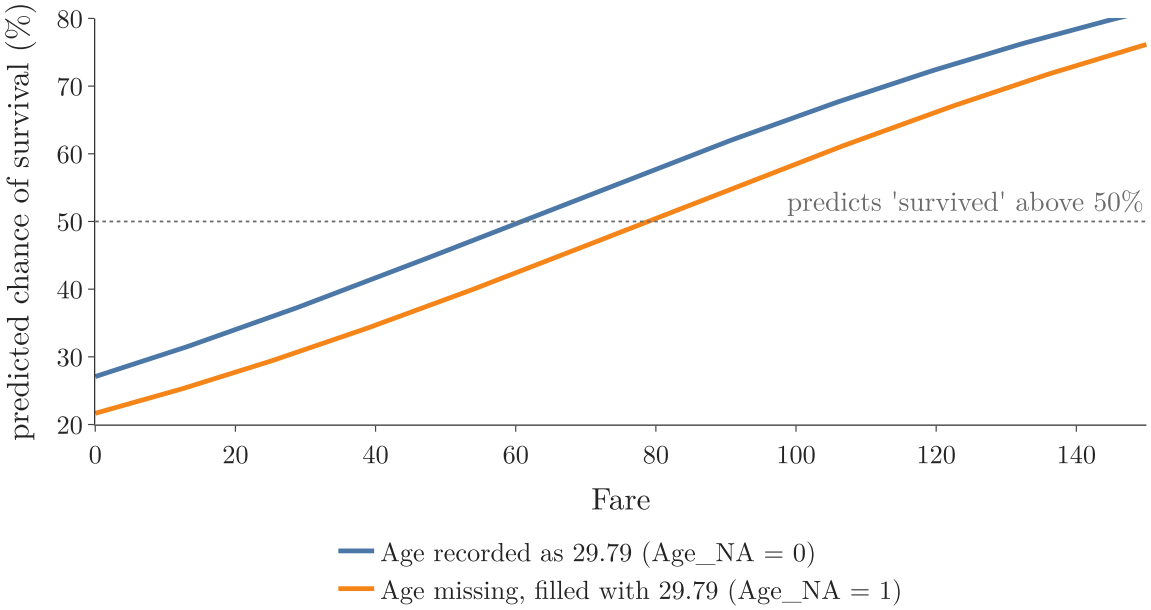



**Figure 8:** Predicted chance of survival against fare, for the same filled age with Age_NA = 0 and Age_NA = 1

Figure 9 shows the model learning that coefficient. Training starts with every coefficient at 0, so both curves are one flat line at 50%. Watch the first iterations: the model learns the fare first, and the two curves rise together as one. From about the sixth iteration the coefficient of `Age_NA` leaves 0, and the dashed orange curve drops below the blue one until the coefficient settles at $-0.30$.

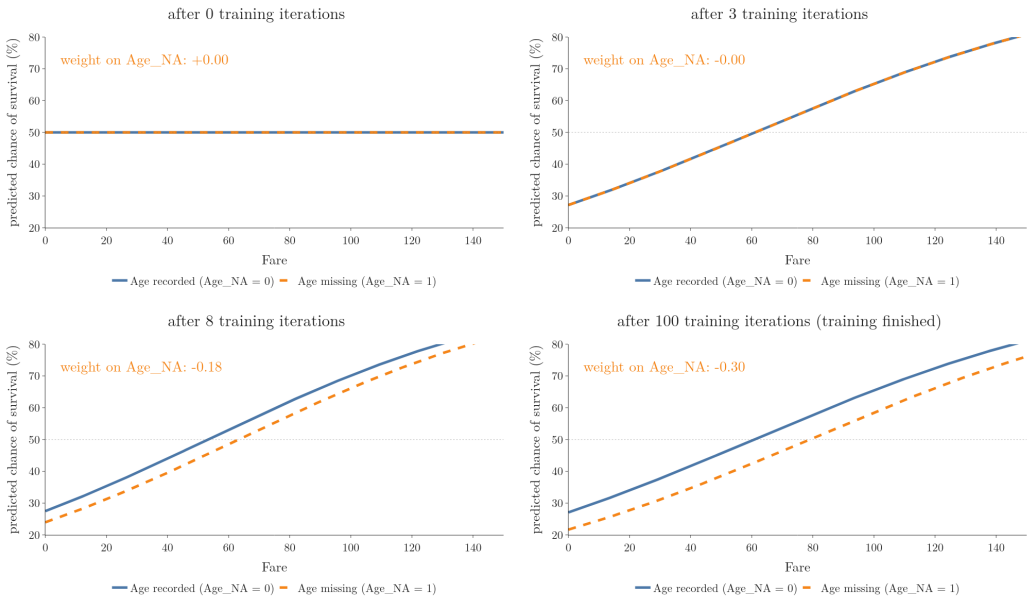



**Figure 9:** The model of Section 6.3 during training: the curves for Age_NA = 0 and Age_NA = 1 start as one and separate as the coefficient of Age_NA moves from 0 to -0.30

**Extra:** One test set of 179 passengers is a small **sample** (G-140), so the Notebook also averages over 100 different splits of all 891 passengers (5-fold cross-validation, repeated 20 times). The mean accuracy is 65.7% without the indicator and 66.3% with it. The gain is small because the model has only two features, but it holds on average.

**Python:** scikit-learn's **`MissingIndicator`** (G-119).

```
from sklearn.impute import MissingIndicator

mi = MissingIndicator()
mi.fit(X_train)                    # which columns have gaps?
mi.features_                       # array([0]): only Age
X_train_missing = mi.transform(X_train)
X_test_missing = mi.transform(X_test)
```

**`features_`** lists the positions of the columns that had gaps during `fit`. `transform` returns one True/False column for each of them. We then add it to the table, for example `X_train.assign(Age_NA=X_train_missing.ravel())`, and impute as before.

### 6.4 The shortcut: `add_indicator=True`

**Key point:** `SimpleImputer(add_indicator=True)` imputes and adds the indicator columns in one step.

The **missing indicator** (G-1233) is common enough that `SimpleImputer` has a parameter for it (the parameters of SimpleImputer list it). With **`add_indicator=True`** (G-58), the imputer fills the gaps and appends one indicator column for each column that had gaps.

**Python:** Imputation and indicator in one object.

```
si = SimpleImputer(add_indicator=True)
X_train_trf = si.fit_transform(X_train)
X_test_trf = si.transform(X_test)
si.get_feature_names_out()
# ['Age', 'Fare', 'missingindicator_Age']
```

The result is the same three columns, and **logistic regression** (G-1120) again reaches 63.1%. `add_indicator=True` is the easier route: no separate object, and no joining of columns by hand.

Use the `MissingIndicator` class with an imputer that has no such parameter, such as the **random sample imputation** (G-1616) of Section 2. Otherwise, `add_indicator=True` is simpler.

## 7. Choosing the imputer automatically with grid search

**Key point:** Put the imputers inside a **pipeline** (G-1499), and let `GridSearchCV` try every strategy and keep the one with the best cross-validated score.

### 7.1 The idea

**Key point:** The imputation strategy is a setting like any other, so grid search can tune it.

Should `Age` get the mean or the median? Should a categorical gap get the mode or a constant? Instead of deciding by hand, we can try every combination and measure which gives the best model. A setting chosen before training, such as the imputation strategy, is a **hyperparameter** (G-910).

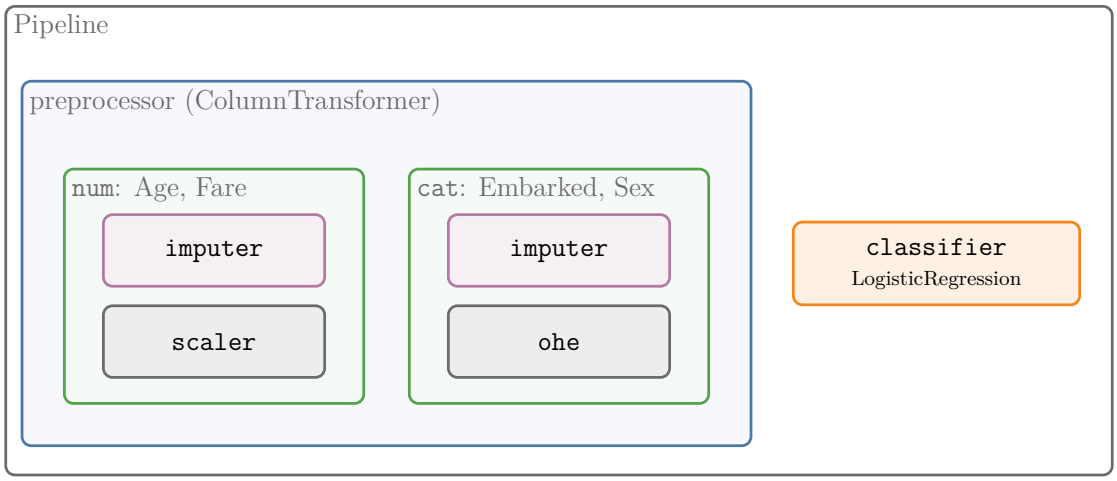
**Grid search** (G-872) does this: it trains the model once for every combination of the settings we list, scores each with **cross-validation** (G-510), which trains and tests several times on different parts of the training data, and keeps the best. For example, with 10 parts: cut the training set into 10 equal parts, train on 9 of them and score on the tenth, do this 10 times so that each part is the scoring part once, and average the 10 scores. scikit-learn's `GridSearchCV` (G-89) runs it (see hyperparameter tuning with a pipeline). Grid search can tune the imputer only if the imputer is part of the model, so we build one **pipeline** (G-1499), a single object that bundles the processing steps and the model, from the raw data to the prediction.

### 7.2 The pipeline

**Key point:** Numerical columns are imputed and scaled, categorical columns are imputed and one-hot encoded, then **logistic regression** (G-1120) is trained.

We use the Titanic file again, dropping `PassengerId`, `Name`, `Ticket` and `Cabin`. In the training set, `Age` has 148 gaps and `Embarked`, the port where the passenger boarded, has 2.

Figure 10 shows the **pipeline** (G-1499). Each step has a name, shown in typewriter font.




`preprocessor__num__imputer__strategy` = `strategy` of the imputer in `num`

**Figure 10:** The nested pipeline; a setting's full name joins the step names with double underscores

**Python:** The pipeline of Figure 10.

```
num = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())])
cat = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("ohe", OneHotEncoder(handle_unknown="ignore"))])
preprocessor = ColumnTransformer(transformers=[
    ("num", num, ["Age", "Fare"]),
    ("cat", cat, ["Embarked", "Sex"])])
clf = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression())])
```

`handle_unknown="ignore"` turns a category never seen in training into all zeros instead of an error (**one-hot encoding** (G-1379)).

**Extra:** A `ColumnTransformer` drops every column it is not told about (`remainder="drop"` by default). So this model uses only `Age`, `Fare`, `Embarked` and `Sex`; `Pclass`, `SibSp` and `Parch` are silently left out. Add them to a list, or set `remainder="passthrough"`, to use them.

### 7.3 Naming the settings

> **Key point:** A setting deep inside a **pipeline** (G-1499) is named by its path: step names joined by two underscores, then the parameter name.

A setting inside a pipeline is named `step__parameter` (see hyperparameter tuning with a pipeline). In a nested pipeline the name is the whole path from the outer pipeline down, joined by `__`. Figure 10 shows `preprocessor__num__imputer__strategy`: the `strategy` of the `imputer` in `num`, inside `preprocessor`.

The grid lists the values to try for each name:

| Setting | Values tried |
|---|---|
| `preprocessor__num__imputer__strategy` | `"mean"`, `"median"` |
| `preprocessor__cat__imputer__strategy` | `"most_frequent"`, `"constant"` |
| `classifier__C` | 0.1, 1, 10, 100 |

`C` controls how strongly **logistic regression** (G-1120) is held back from fitting the training data too closely (the strength C). The grid has 16 combinations:

$$2 \times 2 \times 4 = 16$$

> **Python:** The **grid search** (G-872).
```
param_grid = {
    "preprocessor__num__imputer__strategy": ["mean", "median"],
    "preprocessor__cat__imputer__strategy":
        ["most_frequent", "constant"],
    "classifier__C": [0.1, 1.0, 10, 100],
}
grid_search = GridSearchCV(clf, param_grid, cv=10)
grid_search.fit(X_train, y_train)
grid_search.best_params_
```
`clf.get_params()` lists every valid name, if we are unsure of a path. With `cv=10`, each of the 16 combinations is trained 10 times, 160 fits in all, each on nine tenths of the training set and scored on the other tenth.

### 7.4 The result where the fill matters: house prices

> **Key point:** When the feature with gaps drives the **target** (G-1949), grid search finds a clear winner: on house prices, the "Missing" category cuts the average error from 53,400 to 46,400 dollars.

Grid search can only tell two imputers apart if the **feature** (G-772) they fill matters for the target. Think of two ways to fill a blank on a form: the choice matters only if someone reads that line. The house data of Section 5 meets that condition. `FireplaceQu` has 47.7% gaps in

the training set, and those gaps mark cheaper houses (Section 5.3). We predict `SalePrice` from `GarageQual` and `FireplaceQu` with a three-step **pipeline** (G-1499): impute, one-hot encode (one column per category), then linear regression (fitting a line). The grid tries the two categorical strategies.

The score is the **mean absolute error** (G-1194): the average size of the gap between the predicted and the real price, in dollars. Lower is better.

| `imputer__strategy` | Mean CV error |
|---|---|
| `"most_frequent"` | 53,447 dollars |
| `"constant"` (a "missing" category) | 46,382 dollars |

**`best_params_`** (G-66) picks `"constant"`, and the refitted pipeline's error on the test set is 47,272 dollars. The reason is the price shift of Section 5.3, seen from the other side. Mode imputation turns the 557 cheap houses with no fireplace into Gd houses, so the model can no longer tell them apart from real Gd houses. A category of their own keeps them as a separate group with its own, lower price. The scikit-learn example on imputing before an estimator makes the same point: the choice of imputer changes the model's score, so it is worth comparing (scikit-learn docs, Imputing missing values before building an estimator).

**Python:** The **grid search** (G-872) on the house data.

```
house_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("ohe", OneHotEncoder(handle_unknown="ignore")),
    ("model", LinearRegression())])
house_search = GridSearchCV(house_pipe,
    {"imputer__strategy": ["most_frequent", "constant"]},
    cv=10, scoring="neg_mean_absolute_error")
house_search.fit(Xh_train, yh_train)
house_search.best_params_    # {'imputer__strategy': 'constant'}
```

**`scoring="neg_mean_absolute_error"`** scores each setting by minus its mean absolute error, because grid search always keeps the highest score. With strings, `"constant"` fills the gaps with the word `"missing_value"` by default.

**Extra:** Averaged over 100 splits of all 1,460 houses (10-fold **cross-validation** (G-510), repeated 10 times), the error is 54,015 dollars with the mode and 46,863 dollars with the constant category.

### 7.5 When every imputer ties: the Titanic pipeline

**Key point:** On the Titanic data, all four imputer combinations tie at 78.8%, because the **feature** (G-772) with gaps, `Age`, barely affects the predictions.

The Titanic **pipeline** (G-1499) of Section 7.2 shows the other side of the condition.

`best_params_` reports `C = 1`, `"most_frequent"` for the categorical features and `"mean"` for the numerical ones, with a cross-validated accuracy of 78.8%. The full table, **`cv_results_`** (G-73), shows that only `C` changes the score:

| `C` | Imputer strategies | Mean CV accuracy |
|---|---|---|
| 1, 10 or 100 | any of the 4 combinations | 78.8% |
| 0.1 | any of the 4 combinations | 78.6% |

Figure 11 puts both searches side by side. Watch the left panel, where the fill changes the error by 7,000 dollars, against the right panel, where every row is identical and only the column `C` changes the score.

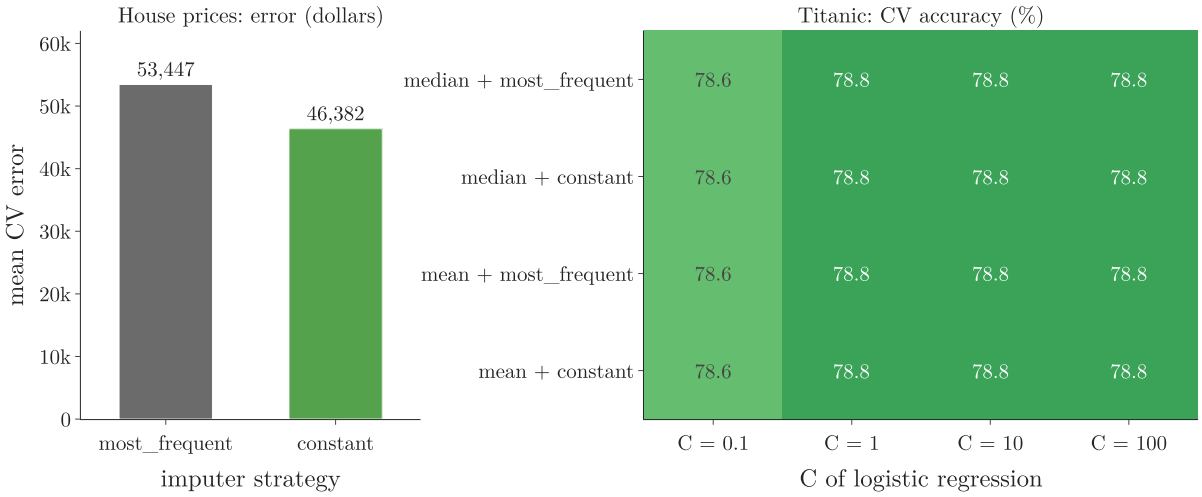



**Figure 11:** Grid search results: on house prices the "Missing" category clearly wins; on the Titanic pipeline all four imputer combinations tie and only C matters

The four imputer combinations score exactly the same, so grid search reports the first of the tied combinations. The categorical fills touch only the 2 **observations** (G-1374) with no `Embarked`, and `Age` hardly matters (next Extra).

> **Extra:** The experiment behind the tie (with `C = 1`, the same 10 folds; the last cell of the Notebook).
>
> | Change to the pipeline | Mean CV accuracy |
> |---|---|
> | `Age` filled with the mean or the median | 78.8% |
> | `Age` filled with 0 | 78.8% |
> | `Age` filled with 99 | 78.9% |
> | `Age` removed from the inputs | 78.8% |

Removing `Age` entirely gives the same score, fold for fold, so no fill of `Age` can change it; only the extreme fill of 99 flips a few predictions.

**Extra: Grid search** (G-872) fits only on the training set; the test set stays unseen. After the search, `grid_search` refits the best pipeline on the whole training set, and we score it on the test set once: `grid_search.score(X_test, y_test)` gives 76.0%. The test score is the number to report, not the 78.8% from **cross-validation** (G-510), because the settings were chosen to maximise that one, which makes it too optimistic (Cawley and Talbot 2010).

## 8. Summary

| | Random sample imputation | Missing indicator | Grid search over imputers |
|---|---|---|---|
| What it does | fills each gap with a random known value | adds a True/False column per column with gaps | tries every imputer setting, keeps the best |
| Column types | numerical and categorical | any | any |
| Keeps the distribution | yes | does not fill; used with another imputer | depends on the chosen imputer |
| Main risk | weaker links to other columns; randomness; training data needed in production | an extra column that may carry no information | slow when the grid is large |
| In scikit-learn | no; done in pandas with `sample` | `MissingIndicator`, or `SimpleImputer(add_indicator=True)` | `GridSearchCV` over a `Pipeline` |
| On this data | `Age`: variance 204.35 to 200.03 | accuracy 61.5% to 63.1% | house prices: "Missing" beats mode, error 46,400 vs 53,400 dollars |

- **Random sample imputation** (G-1616) draws each fill value from the feature's own known values, so the shape and variance stay almost the same. The technique suits linear models, because filling with one value shrinks the variance and biases the coefficients a linear model learns, while random draws keep the variance.
- Random sample imputation weakens the feature's **covariance** (G-496) with others (`Age` with `Fare`: 71.51 to 53.26), because each drawn value ignores the rest of the row, and needs the training values at prediction time, because gaps in new data are filled from the training column.
- In production, seed each draw with a value from the row, so the same input always gets the same fill and the same prediction, and the user does not get two answers to one question.
- On categorical data, check the category shares and the **target** (G-1949) per category. In `FireplaceQu` the shares held, but the prices per category shifted badly: too many values were missing.
- A missing indicator lets the model learn whether a missing value is informative: on the Titanic data, 28.4% of passengers with no age survived against 39.2% with one, which **mean imputation** (G-1197) alone hides. The indicator is added next to any imputation.
- Grid search can tune the imputation strategy along with the model, if the imputers sit inside the **pipeline** (G-1499). Settings are named by their path, joined with `__`. On the house prices, **grid search** (G-872) picked the "Missing" category, which cut the

average error from 53,400 to 46,400 dollars, because a category of their own keeps the cheap houses with no fireplace apart from real Gd houses. On the Titanic data every imputer tied, because `Age` barely affects the predictions.
- So random sample imputation keeps a column's shape, a **missing indicator** (G-1233) keeps the fact that a value was missing, and grid search picks the imputer for us when the choice matters.

## 9. Sources

### Built from

- CampusX, "Missing Indicator | Random Sample Imputation | Handling Missing Data Part 4", YouTube, https://www.youtube.com/watch?v=Ratcir3p03w

### Other references

- van Buuren, S. (2018). *Flexible Imputation of Missing Data*, 2nd edition. Chapman & Hall/CRC. Free online at stefvanbuuren.name/fimd. §1.3.3 "Mean imputation" (underestimates the variance, disturbs the relations between variables, biases almost any estimate other than the mean).
- scikit-learn examples, *Imputing missing values before building an estimator* (Impute section of the example gallery).
- Cawley, G. C. and Talbot, N. L. C. (2010). On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research* 11, 2079–2107.

## 10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

| Term | Meaning |
|---|---|
| Grid search (G-872) | Training a model for every combination of listed settings and keeping the best by cross-validation. |
| Random sample imputation (G-1616) | Filling each gap with a value drawn at random from the column's known values. |
| Missing indicator (G-1233) | A 0/1 (True/False) column recording whether a value was missing, added next to the imputed column so the model can learn whether being missing itself matters. |
| Tree-based algorithm (G-2013) | An algorithm that splits the data with simple conditions; hardly affected by outliers. |
| `sample(n)` (G-140) | The pandas method that draws `n` values at random from a Series or DataFrame. |

| Term | Meaning |
|---|---|
| `random_state` (G-128) | A seed that fixes a random draw, so the same code gives the same result. |
| Per-row seed (G-1479) | A seed taken from a row's own values, so the same input always gets the same random fill. |
| `MissingIndicator` (G-119) | The scikit-learn class that builds columns marking which values were missing (missing indicators); `features_` lists the columns with gaps. |
| `add_indicator=True` (G-58) | The `SimpleImputer` setting that fills the gaps and also adds columns marking which values were missing (missing indicators), in one step. |
| `best_params_` (G-66) | The best combination of settings found by `GridSearchCV`. |
| `cv_results_` (G-73) | The scores of every combination tried by `GridSearchCV`. |
| Feature (G-772) | One piece of information about each example that a model uses (e.g. a student's CGPA). |
| Target (G-1949) | The output column we predict, such as the class; also called the label. |
| Observation (G-1374) | One record of a dataset: one row of the data table. |
| Univariate imputation (G-2051) | Filling the gaps in a column (imputation) using only that column's other values. |
| Distribution (G-626) | How a column's values spread over their range. |
| Variance (of data) (G-2074) | The average squared distance of the values from their mean; the square of the standard deviation; divide by $n$ for a population (NumPy default) or $n - 1$ for a sample (pandas default). |
| Mean imputation (G-1197) | Filling every gap with the mean of the column's known values. |
| Logistic regression (G-1120) | A classification algorithm that passes a weighted sum of the inputs through the sigmoid to give the probability of a class; the line (or plane) where that probability is 0.5 is the boundary it learns between the classes. |
| Covariance (G-496) | A number that says how two numerical features move together: positive if they rise together, negative if one rises as the other falls. Its size depends on the units. |

| Term | Meaning |
|---|---|
| Random seed (G-1617) | A number that fixes a random number generator so that a run can be repeated exactly. |
| Data leakage (G-535) | Information from the test rows reaching the model during training, for example when a scaling or feature-selection step is fitted on all the data before the split; the test scores then come out higher than the model will really do on new data. |
| Accuracy (G-162) | The fraction of predictions that are correct. |
| MAR (G-1158) | Missing at random: the gaps depend on another, recorded column. |
| Coefficient ($\beta_i$) (G-407) | The weight of one input column: the change in the output per unit of that input, others fixed. |
| Sigmoid function (G-1798) | An S-shaped function that squashes any number into the range 0 to 1, $\sigma(z) = 1/(1 + e^{-z})$; it turns a score into a probability, as in logistic regression. |
| Pipeline (G-1499) | One object that chains preprocessing steps and a model, each step's output feeding the next; every input, including new ones after deployment, then gets exactly the same preprocessing. |
| Hyperparameter (G-910) | A setting of an algorithm chosen before training, such as a tree's `max_depth`. |
| Cross-validation (G-510) | Testing a model by training and testing it several times on different parts of the training data. |
| `GridSearchCV` (G-89) | The scikit-learn class that tries every combination of settings in a grid, scores each by cross-validation, and keeps the best. |
| One-hot encoding (G-1379) | Replacing a nominal column by one 0/1 column per category, with a single 1 in each row; also used to represent words. |
| Mean absolute error (MAE) (G-1194) | The average absolute difference between actual and predicted values. |